Data Analytics Project: KC House Sales Dataset Analysis

Dataset Used: kc_house_data.csv
Team Members: Tarun| Sumanth | Sruthi | Subbaraju | Swetha

This document outlines a structured, end-to-end data analysis project based on the King County (KC) House Sales dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyse house prices, bedroom distributions, and pricing patterns using the kc_house_data.csv dataset.

The primary libraries/modules required for this project are:

- Pandas: For all data manipulation, filtering, aggregation, and feature engineering tasks.
- NumPy: Specifically for numerical calculations like price differences (np.diff).
- Visualization Library (e.g., Matplotlib): For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts, Histograms).

Project Coverage:

Category	Skills Covered
Data Pre-processing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (Price Category)
NumPy Calculations	Array conversion, Price difference calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning

Description:

In this scenario we are loading the dataset to the pandas DataFrame and displaying the top 5 rows. After that we are checking for any missing values and if found any, we need to replace the bedrooms and sqft_living columns with mode/mean and price and bathrooms columns with mean of those columns respectively. We also need to convert the type of key columns to numeric if necessary.

Task	Description
1.	Load the KC House Sales dataset using Pandas.
2.	Display the first 5 rows (head()) and column names.
3.	Check for missing values in bedrooms, bathrooms, sqft_living, and price.
4.	Handle missing values: Fill bedrooms with mode, bathrooms/sqft_living/price with mean.
5.	Convert bedrooms, bathrooms, sqft_living, and price columns to numeric if required.

Output:

```
First five Colunm Nmaes
           id             date     price  bedrooms  ...      lat     long  sqft_living15  sqft_lot15
0  7129300520  20141013T000000  221900.0         3  ...  47.5112 -122.257           1340        5650
1  6414100192  20141209T000000  538000.0         3  ...  47.7210 -122.319           1690        7639
2  5631500400  20150225T000000  180000.0         2  ...  47.7379 -122.233           2720        8062
3  2487200875  20141209T000000  604000.0         4  ...  47.5208 -122.393           1360        5000
4  1954400510  20150218T000000  510000.0         3  ...  47.6168 -122.045           1800        7503

[5 rows x 21 columns]
Column Names
 Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
bedrooms       0
bathrooms      0
sqft_living    0
price          0
dtype: int64
bedrooms         int64
bathrooms      float64
sqft_living      int64
price          float64
dtype: object
```

Conclusion:

In this scenario we loaded the CSV file to a pandas DataFrame and displayed the first 5 rows with column names. There are no missing values in the bedrooms, bathrooms, sqft_living, and price. The price and other numeric columns are already in numeric type for requirement.

Scenario 2: Line Graph + Save

Description:

In this scenario, we work with house data to analyze price trends. First, we select only the required columns (id and price) and limit the data to the first 10 records for simplicity. The price values are then converted into a NumPy array to make them suitable for plotting.A line graph is created where the X-axis represents the index (0–9) and the Y-axis represents the house prices. Titles and labels are added to make the graph easy to understand. Finally, the graph is saved as an image file (house_prices_line.png) and displayed

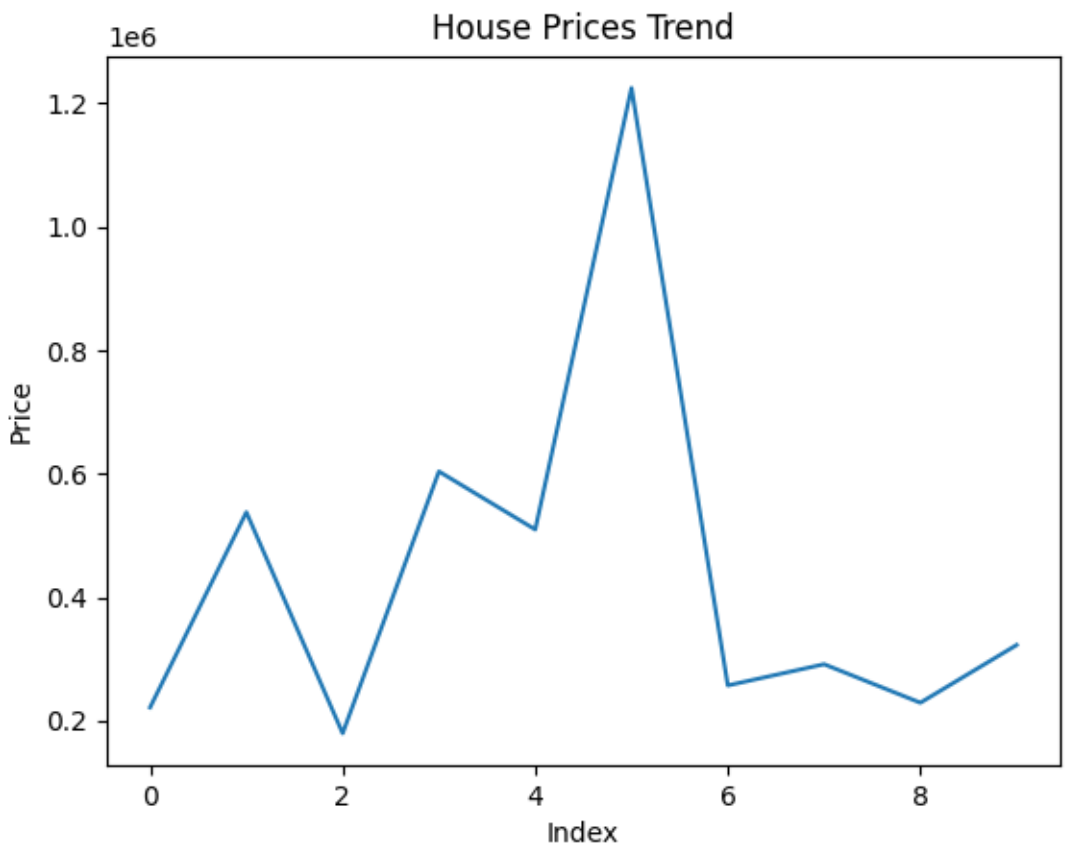
Task	Description
1.	Select id and price columns.
2.	Take first 10 rows only.
3.	Convert price into a NumPy array.
4.	Plot a line graph: X-axis → index (0-9), Y-axis → Price.
5.	Add title and axis labels.
6.	Save the graph: plt.savefig("house_prices_line.png")

Output:

```
First five Colunm Nmaes
           id             date     price  ...     long  sqft_living15  sqft_lot15
0  7129300520  20141013T000000  221900.0  ... -122.257           1340        5650
1  6414100192  20141209T000000  538000.0  ... -122.319           1690        7639
2  5631500400  20150225T000000  180000.0  ... -122.233           2720        8062
3  2487200875  20141209T000000  604000.0  ... -122.393           1360        5000
4  1954400510  20150218T000000  510000.0  ... -122.045           1800        7503

[5 rows x 21 columns]
Column Names
 Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
bedrooms       0
bathrooms      0
sqft_living    0
price          0
dtype: int64
bedrooms         int64
bathrooms      float64
sqft_living      int64
price          float64
dtype: object
```

Graph:

The graph is a line chart where the X-axis represents the index values (0-9) and the Y-axis represents the price of the houses. Each point on the graph shows the price of a house, and the points are connected by a line. The graph helps in visualizing how the prices increase or decrease across the first 10 selected houses.



Conclusion:

This scenario shows how to visualize house price trends using a line graph. By selecting required data, limiting it to 10 records, and converting it into a NumPy array, we make the data ready for plotting. The line graph helps to easily understand how prices change across entries, and adding labels improves clarity. Finally, saving the graph allows it to be used later for reports or analysis.

Scenario 3: Filtering + Bar Chart + Save

Description:

This scenario focuses on analysing expensive houses using data filtering. The dataset is filtered to include only houses with a price greater than 1,000,000 using Pandas. The number of houses is then counted based on the number of bedrooms. The results are converted into NumPy arrays using NumPy. Finally, a bar chart is created using Matplotlib to visualize the distribution.

Task	Description
1.	Filter houses where price > 1,000,000.
2.	Count the number of expensive houses per bedrooms category.
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: X-axis → Bedrooms, Y-axis → Count.
6.	Rotate x-axis labels for clarity.
7.	Save the graph: plt.savefig("expensive_houses_bar.png")

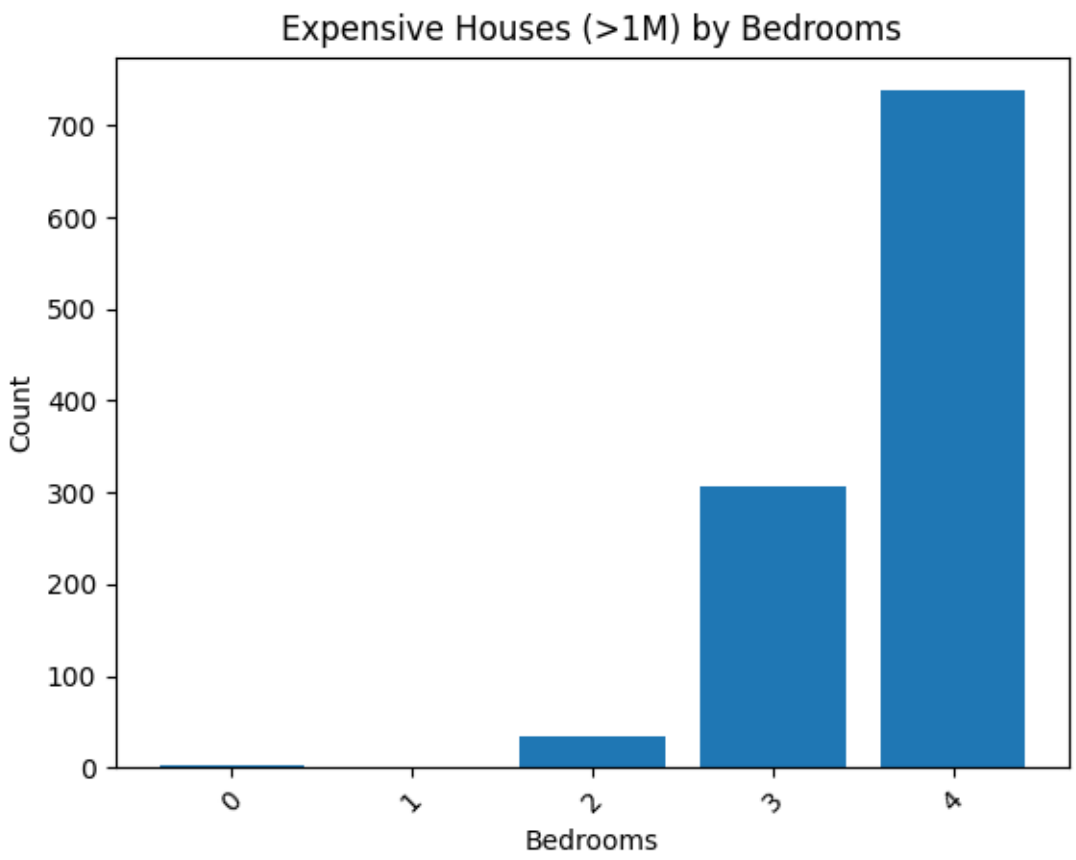
Output:

In this scenario we are counting the total number of expensive houses per bedroom category. We filter the dataset to get houses with price > 1,000,000, then group by bedrooms and plot the bar chart. The bedroom categories with highest expensive house counts are shown below.

```
Name: count, dtype: int64
bedrooms
3    9824
4    6882
2    2760
5    1601
6     272
```

Graph:

The graph is a bar chart where the X-axis represents the number of bedrooms and the Y-axis represents the count of expensive houses. Each bar shows how many high-priced houses belong to a specific bedroom category. Taller bars indicate more expensive houses in that category. Labels on the X-axis may be rotated for better readability.



Conclusion:

The bar chart helps identify which bedroom types are most common among expensive houses. It shows the distribution of high-priced houses clearly. Some bedroom categories have more luxury houses than others. Overall, this helps in understanding housing patterns in the dataset.

Scenario 4: Pie Chart (Bedroom Distribution) + Save

Description:

In this scenario we are analysing the distribution of houses by the number of bedrooms from the dataset. Then we are selecting the top 5 bedroom categories with the highest number of houses and we need to represent those houses graphically using pie chart.

Task	Description
1.	Count the number of houses by bedrooms.
2.	Select top 5 bedroom categories.
3.	Prepare labels and values for the pie chart.
4.	Plot a pie chart using bedroom count as labels and house count as values.
5.	Add percentage labels using autopct (e.g., '%.1f%%').
6.	Save the graph: plt.savefig("bedroom_distribution.png")

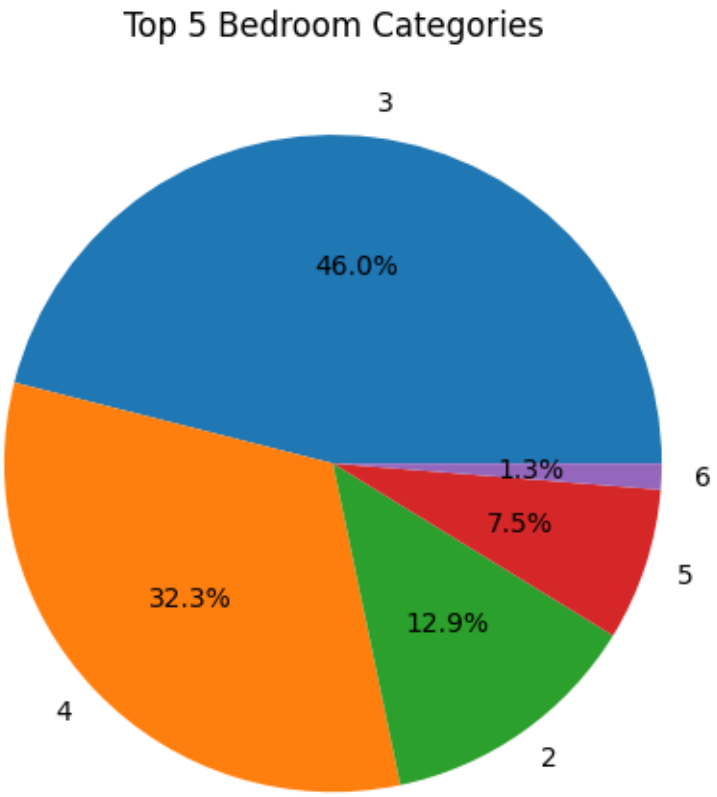
Output:

The total number of houses by the bedroom counts and top 5 category is as shown below.

```
bedrooms
0       13
1      199
2     2760
3     9824
4     6882
5     1601
6      272
7       38
8       13
9        6
10       3
11       1
33       1
Name: count, dtype: int64
bedrooms
3    9824
4    6882
2    2760
5    1601
6     272
Name: count, dtype: int64
```

Graph:

We are using pie chart to visually represent the proportion of houses in the top 5 bedroom categories as shown below.



Conclusion:

In this scenario we are sorting the dataset to get the houses categorised depending on the number of bedrooms. And from the output and the pie chart we can conclude that the 3 bedrooms' house have the highest contribution.

Scenario 5: Advanced Analysis + Multiple Graphs

Description:

This scenario focuses on advanced data analysis of the house dataset using feature engineering, NumPy operations, and visualizations. A new feature “Price Category” is created to classify houses into Affordable, Mid-Range, and Luxury. NumPy is used to analyse price differences for numerical insights. Multiple graphs are generated to understand trends, distributions, and relationships in the data. Finally, statistical analysis is used to derive meaningful business insights.

Task	Description
1.	Create Price Category column: price >= 1,000,000 → "Luxury", 500,000-999,999 → "Mid Range", < 500,000 → "Affordable".
2.	Convert price column to NumPy array.
3.	Calculate price differences using np.diff().
4.	Plot a Line Graph showing price trend for all houses.
5.	Plot a Stacked Bar Chart: count of Price Category per Bedrooms.
6.	Plot a Histogram: distribution of price.
7.	Save graphs: plt.savefig("price_trend.png"), plt.savefig("price_category_stacked.png"), plt.savefig("price_histogram.png")
8.	Identify: Which bedroom category has the most expensive houses, which price category is most common, and the distribution pattern.

Output:

The output summarizes key insights from the house sales data, including price differences, the most expensive bedroom category, and the most common price range.

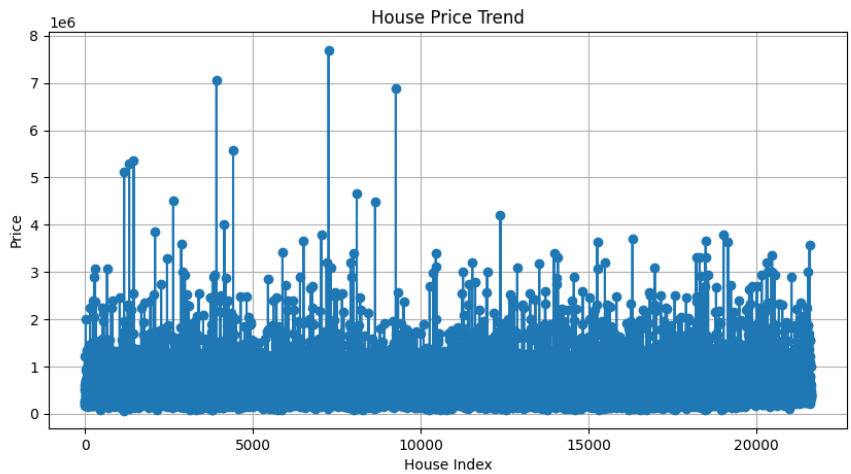
```
l\Python\pythoncore-3.14-64\python.exe c:/Users/DELL/Documents/Python/DataAnalytics/Project_4_House
_sales/main.py
Price differences (first 10): [ 316100. -358000.  424000.  -94000.  715000. -967500.   34350.  -623
50.
   93500.  339500.]

Most expensive bedrooms category:
 bedrooms
8    1.105077e+06
Name: price, dtype: float64

Most common price category:
 Price Category
Affordable    12408
Mid Range      7713
Luxury         1492
Name: count, dtype: int64

Price skewness check:
4.024069144684714
Distribution is Right-skewed (most houses are low price, few very expensive)
PS C:\Users\DELL\Documents\Python\DataAnalytics\Project_4_House_sales>
```

Graphs:

Three graphs have been plotted — a Line graph, Stacked Bar graph, and Histogram — for better visualization.

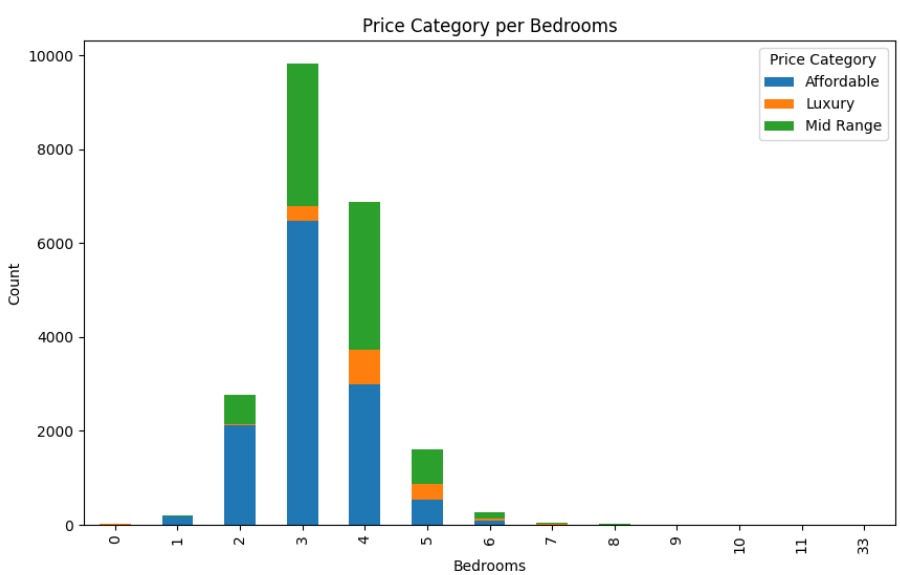
Line Graph

This line graph shows the variation of house prices across all records in the dataset. Each point represents a house price plotted in sequence. It helps visualize how prices fluctuate throughout the dataset. The graph highlights inconsistencies and outliers in pricing.



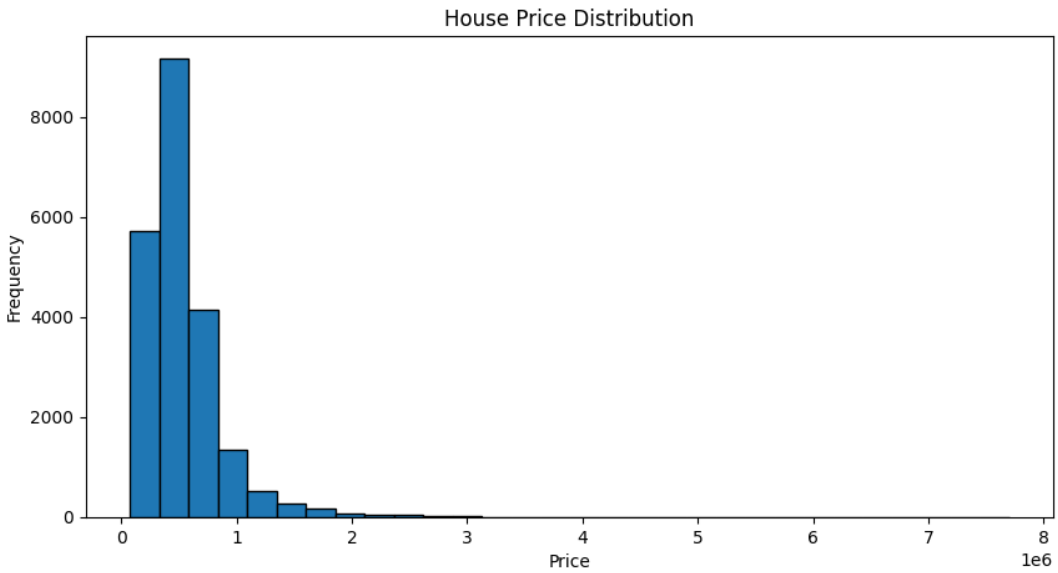
Stacked Bar Chart

This stacked bar chart shows how different price categories are distributed across various bedroom counts. Each bar represents bedrooms, divided into Affordable, Mid-Range, and Luxury segments. It helps compare how house size influences pricing category.



Histogram

This histogram displays the frequency distribution of house prices in different ranges. It shows how many houses fall within each price interval. This helps identify the overall shape of price distribution.



Conclusion:

This scenario provided a complete analysis of the house dataset using feature engineering, NumPy operations, and visualizations. A new “Price Category” feature helped simplify interpretation of house prices into clear segments. Graphical analysis showed how prices vary across houses and revealed important patterns in the data. Most houses fall under affordable and mid-range categories, while luxury houses are limited. Overall, the dataset shows a right-skewed price distribution with a few high-value outliers influencing the market.

Final Conclusion

This project analysed the KC House Sales dataset using Pandas, NumPy, and Matplotlib. It showed that 3-bedroom and 4-bedroom homes dominate the housing market, with the majority of houses falling in the Affordable price range (below $500,000). The Luxury segment is small but concentrated in larger homes with 4-5 bedrooms. The data visualizations helped clearly understand patterns in house pricing and bedroom distribution. Overall, the analysis gave useful insights into the King County real estate landscape.

What We Have Learned

In this project, we have learnt how to clean and process real-world housing data using Pandas and perform numerical calculations using NumPy. We have also learnt how to create different visualizations like line graphs, bar charts, pie charts, stacked bar charts, and histograms using Matplotlib. Most importantly, we understood how to turn raw real estate data into meaningful insights using an end-to-end data analysis process.